

Session 7: Constraints & Views
Trainer Coverage: Constraints: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY. Creating and querying VIEWS. Example: Create “ActiveUserView” showing users with >5 transactions. Pending

Tasks

- 1) Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.

Query:

Run SQL query/queries on table my database.restaurant: 

```
1 CREATE TABLE Restaurant (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(100) NOT NULL,  
4     location VARCHAR(150) NOT NULL,  
5     cuisine VARCHAR(100)  
6 );
```

2) Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).

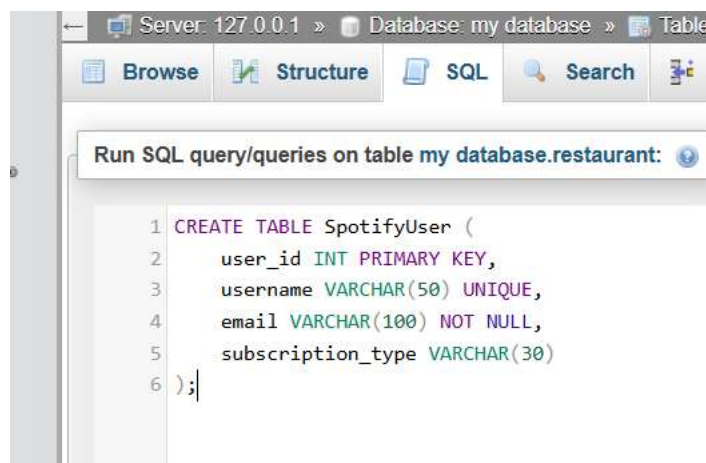
A screenshot of a SQL query editor interface. At the top, there is a button labeled "Run SQL query/queries on table my database.restaurant:". Below the button, the SQL code is displayed in a monospaced font with syntax highlighting. The code is as follows:

```
1 CREATE TABLE FoodOrder (  
2     order_id INT PRIMARY KEY,  
3     restaurant_id INT,  
4     user_id INT,  
5     order_total DECIMAL(10,2),  
6     FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
7 );
```

3) Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL.

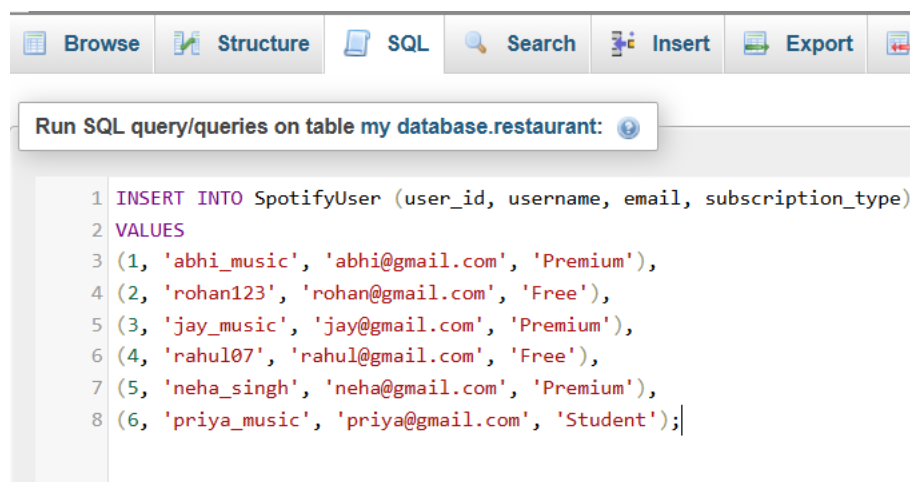
Hint: Use the UNIQUE and NOT NULL constraints when creating the table.

Query:



The screenshot shows a database management tool interface with a toolbar containing 'Browse', 'Structure', 'SQL', and 'Search' buttons. Below the toolbar, a text box says 'Run SQL query/queries on table my database.restaurant:'. The SQL editor displays the following code:

```
1 CREATE TABLE SpotifyUser (  
2     user_id INT PRIMARY KEY,  
3     username VARCHAR(50) UNIQUE,  
4     email VARCHAR(100) NOT NULL,  
5     subscription_type VARCHAR(30)  
6 );|
```



The screenshot shows the same database management tool interface. The toolbar now includes an additional 'Insert' button. The text box still says 'Run SQL query/queries on table my database.restaurant:'. The SQL editor displays the following code:

```
1 INSERT INTO SpotifyUser (user_id, username, email, subscription_type)  
2 VALUES  
3 (1, 'abhi_music', 'abhi@gmail.com', 'Premium'),  
4 (2, 'rohan123', 'rohan@gmail.com', 'Free'),  
5 (3, 'jay_music', 'jay@gmail.com', 'Premium'),  
6 (4, 'rahul07', 'rahul@gmail.com', 'Free'),  
7 (5, 'neha_singh', 'neha@gmail.com', 'Premium'),  
8 (6, 'priya_music', 'priya@gmail.com', 'Student');|
```

Output:

☐ Show all

Number of rows: 25

Filter rows:

Sort by key

Extra options

			▼	user_id	username	email	subscription_type
☐		Edit		Copy		Delete	1 abhi_music abhi@gmail.com Premium
☐		Edit		Copy		Delete	2 rohan123 rohan@gmail.com Free
☐		Edit		Copy		Delete	3 jay_music jay@gmail.com Premium
☐		Edit		Copy		Delete	4 rahul07 rahul@gmail.com Free
☐		Edit		Copy		Delete	5 neha_singh neha@gmail.com Premium
☐		Edit		Copy		Delete	6 priya_music priya@gmail.com Free

4) Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

Run SQL query/queries on table my database.spotifyus:

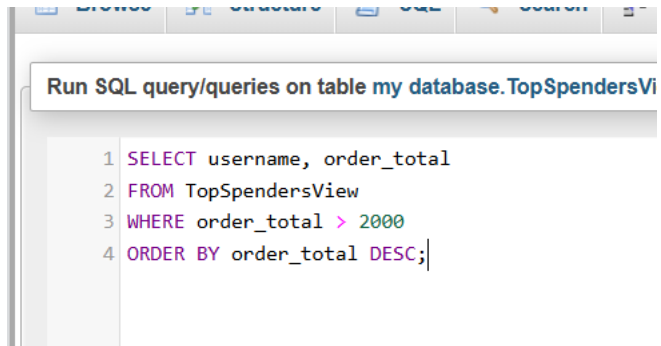
```
1 CREATE VIEW TopSpendersView AS
2 SELECT
3     s.username,
4     f.order_total
5 FROM FoodOrder f
6 JOIN SpotifyUser s
7     ON f.user_id = s.user_id
8 WHERE f.order_total > 1000;
```

Output:

☐ Profiling	[Edit inline]	[Edit]	[Expla
username	order_total		
Query results operations			

5) Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending.

Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.



```
Run SQL query/queries on table my database.TopSpendersVi

1 SELECT username, order_total
2 FROM TopSpendersView
3 WHERE order_total > 2000
4 ORDER BY order_total DESC;
```